

# The Bottleneck Moves Twice: Three Regimes of On-Device LLM Latency

Bapista Khan  
Collab-Foundry  
bapistakhan@gmail.com

## ABSTRACT

On-device language models are deployed for privacy, offline operation and cost, yet the latency intuition the community carries is inherited from datacentre GPU serving, where autoregressive *decode* dominates. Prior work has characterised inference on datacentre CPUs with matrix accelerators and HBM, and quantisation effects on high-end Apple Silicon. We ask a question neither answers: **across the commodity hardware an organisation actually owns, how much of latency behaviour is a property of the machine, and how much is a property of the workload?** We instrument end-to-end inference across four on-device platforms—two x86 CPU vendors, an ARM single-board computer, and 8 GB Apple Silicon—and find that **the prefill share of end-to-end latency converges to within two percentage points at a given context length across all non-accelerated devices**, despite absolute latency differing by up to 14× and model size by 4.7×. At ~1,400 prompt tokens every such device sits at 91–94% prefill; by ~2,440 tokens, 96–97%. A same-model control isolates the effect: identical weights, 14.4× apart in wall-clock, two points apart in share. The invariance breaks in exactly one circumstance. When weights plus KV cache exhaust memory, the device re-inverts to decode-bound and throughput collapses—49 to 17 tok/s on 8 GB Apple Silicon beyond ~2,029 tokens. **How many regimes a deployment traverses is therefore set by memory headroom**, a design-time property readable from specifications. Together these make on-device latency **predictable before execution**: the regime follows from the prompt, and the regime boundary follows from the hardware. We argue this is the missing precondition for regime-aware scheduling across heterogeneous owned devices, and that single-number throughput reporting conceals both properties. Harnesses and raw data are released.

## KEYWORDS

on-device inference, mobile systems, LLM, latency, prefill, memory wall, measurement

## 1 INTRODUCTION

Running language models on the device rather than in the datacentre is no longer exotic. Privacy requirements, offline operation, and the cost of inference at scale have pushed capable models onto phones, laptops and edge hardware [10, 14, 15]. The engineering intuition that travelled with them did not come from those devices. It came from GPU serving, where the dominant cost is autoregressive *decode* and where optimisation effort—batching, paged attention, speculative decoding—has been directed accordingly [7, 8].

We took that intuition to two on-device platforms and measured it. It does not merely fail. It fails *differently at different context lengths on the same machine*.

On an 8 GB Apple Silicon device, a 3B model is **decode-bound** at short context—prefill accounts for only 20% of end-to-end latency. As context grows it becomes **prefill-bound**, peaking at 80% near 1,500 tokens. Then, past roughly 2,029 tokens, it **re-inverts**: prefill’s share falls back to ~69% while decode throughput collapses from 49 to 17 tok/s, as weights plus a growing KV cache exhaust unified memory. Nothing changed but the length of the prompt.

Smaller models on the same device never reach that third regime—0.5B and 1.5B climb monotonically and their decode throughput declines only gently, because their footprint leaves headroom. A CPU-only machine sits at the other extreme: across the whole sweep it never leaves the prefill-bound regime, climbing monotonically from 38% to 95%.

So the number of regimes a deployment traverses is not a property of the platform alone. It is set by the **headroom between model footprint and memory capacity**—which makes it something a developer chooses at design time, usually without knowing they are choosing it.

### Contributions.

- We identify **three distinct latency regimes** for on-device inference and show a single device traversing all three as context grows (§2).
- We show which regimes a configuration reaches is governed by memory headroom, not platform class (§2), and give the mechanism for both transitions (§3).

**Table 1: The three regimes. The binding constraint changes, and with it the only lever that helps.**

| Regime           | Signature                                     | Bound by        | Lever          |
|------------------|-----------------------------------------------|-----------------|----------------|
| 1. Decode-bound  | prefill <50%                                  | sequential cost | output length  |
| 2. Prefill-bound | prefill rising, >50%                          | compute         | context length |
| 3. Memory-bound  | prefill <i>falling</i> ,<br>decode collapsing | capacity        | KV growth      |

**Table 2: One device, three regimes. Qwen2.5-3B (4-bit) on an 8 GB Apple Silicon machine: prefill share rises, peaks, then reverses as decode collapses past the memory wall. Means over five repetitions.**

| Context (tokens) | Prefill (% of latency) | Decode (tok/s) | Regime            |
|------------------|------------------------|----------------|-------------------|
| 55               | 19.9                   | 49.5           | 1 · decode-bound  |
| 140              | 35.4                   | 49.2           | 1                 |
| 300              | 50.5                   | 49.2           | → 2 · crossover   |
| 600              | 64.6                   | 49.2           | 2 · prefill-bound |
| 1000             | 74.5                   | 48.4           | 2                 |
| 1500             | <b>79.9</b>            | 43.5           | 2 · peak          |
| 2000             | 68.8                   | <b>23.5</b>    | → 3 · memory wall |
| 2336             | 69.4                   | <b>17.3</b>    | 3 · memory-bound  |

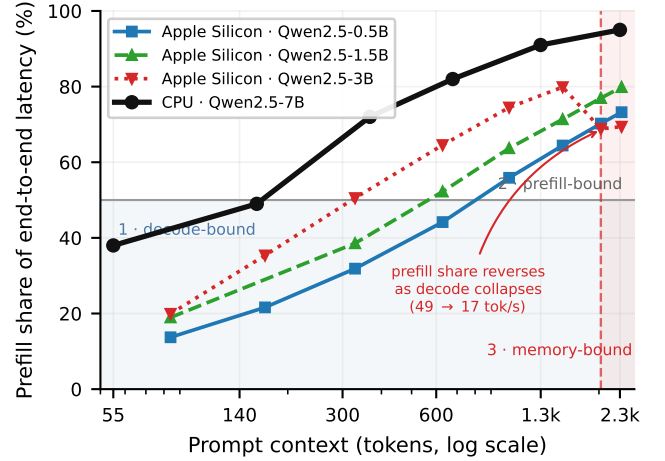
- We draw out what follows for benchmarking, scheduling and context budgeting, and argue that single-number throughput reporting conceals every transition (§4).
- We release both harnesses and all raw data (§6).

## 2 THREE REGIMES

*Setup.* Both studies sweep prompt length from 55 to 2,336 tokens with five repetitions per point, discard warm-up runs, and instrument prefill and decode separately end to end. Platform A is a CPU-only machine (AMD Ryzen 7 8845HS, 8 cores / 16 threads, 14 GiB) running quantized Qwen2.5-7B, Llama-3.1-8B and Gemma2-9B through `llama.cpp` [6]. Platform B is an 8 GB MacBook Air running 4-bit MLX builds of Qwen2.5-Instruct at 0.5B, 1.5B and 3B—a deliberate size sweep on a device where larger families do not comfortably fit, which turns out to be the point. Every number traces to a released raw-results file.

Table 2 is the paper. A developer who profiles this configuration at 600 tokens and one who profiles it at 2,300 tokens will reach opposite conclusions about what to optimise, on identical hardware running an identical model.

*Headroom decides how far you travel.* On the same device, Qwen2.5-0.5B climbs monotonically from 13.7% to 73.2% prefill with decode declining only gently (250 to

**Figure 1: Prefill’s share of end-to-end latency against context length. The CPU (black) climbs monotonically to 95%: it never leaves regime 2. On Apple Silicon the smaller models climb and level off, while Qwen2.5-3B peaks near 1,500 tokens and then reverses—not because prefill became cheap, but because decode collapsed past the memory wall. Means over five repetitions; one flagged 1.5B outlier excluded.**

208 tok/s), and 1.5B behaves the same way (19.0% to 79.9%; 96 to 87 tok/s).<sup>1</sup> Neither reaches regime 3, because neither exhausts unified memory. Only the 3B model—the one whose weights plus KV cache approach the 8 GB ceiling—re-inverts.

*Some platforms never leave regime 2.* Platform A shows no reversal anywhere in the sweep. Prefill climbs monotonically—38% at 55 tokens, 72% at 367, 91% at 1,299, **95%** at 2,336—while decode sits flat at 11–12 tok/s. With 14 GiB and a 7B model there is ample headroom, so memory never binds; the machine simply lacks the parallelism to make prefill cheap. Llama-3.1-8B and Gemma2-9B reproduce this at 95–96% prefill, and the effect reproduces on a second CPU vendor (Intel Comet Lake), where it is if anything stronger.

*The crossover between regimes 1 and 2 also moves with model size,* and in the direction that surprises people: *earlier* for larger models—~1,029 tokens at 0.5B against ~329 at 3B. Bigger models reach the prefill-bound regime sooner and the memory-bound regime sooner.

*Quantization does not behave as the folklore says.* On Platform A prefill *speed* is non-monotonic in footprint: the 8-bit build prefills fastest and an intermediate K-quant slowest, inverting the “quantize down for speed” heuristic [4, 5, 9].

<sup>1</sup>One anomalous 1.5B point near 140 tokens is excluded, as in the underlying study.

Smaller is not reliably faster. What smaller reliably buys is *headroom*—and headroom is what keeps a deployment out of regime 3.

### 3 WHY THE BOTTLENECK MOVES

#### 3.1 Why the share is invariant

The invariance of Section 2 is not a coincidence; it follows from the definition of the quantity. Write  $N_p$  for prompt tokens,  $N_o$  for generated tokens, and  $R_p, R_d$  for the platform’s prefill and decode rates in tokens per second. The prefill share of end-to-end latency is

$$\phi = \frac{N_p/R_p}{N_p/R_p + N_o/R_d} = \frac{1}{1 + \rho \frac{N_o}{N_p}}, \quad \rho \equiv \frac{R_p}{R_d}. \quad (1)$$

Only two quantities appear: the workload shape  $N_o/N_p$ , and  $\rho$ , the *ratio* of the two rates. Absolute speed does not. If one platform is  $k$  times faster than another at everything— $R_p \rightarrow kR_p$  and  $R_d \rightarrow kR_d$ —then  $\rho$  is unchanged and so is  $\phi$ . Two machines whose wall-clock differs by an order of magnitude therefore report the same share, which is precisely what Section 2 measures: a 14× spread in latency and a two-point spread in  $\phi$ .

Equation 1 also says when the invariance *must* fail: whenever  $\rho$  is not constant across platforms. That gives two predictions, and the rest of this section is their two instances.

*Prediction 1: accelerators that parallelise prefill.* A GPU raises  $R_p$  far more than  $R_d$ , since prefill is parallel across tokens while decode is sequential and bandwidth-bound. Larger  $\rho$  means smaller  $\phi$ : the platform looks decode-bound. *This is the datacentre intuition, and Equation 1 reproduces it as a special case rather than a contradiction.* We measure no accelerated device here, so we claim invariance only across non-accelerated platforms.

*Prediction 2: collapse of  $R_d$ .* If decode throughput falls sharply while prefill is unaffected,  $\rho$  rises and  $\phi$  turns back down—not because prefill got cheaper, but because decode got worse. The regime 2  $\rightarrow$  3 transition below is exactly this.

Two transitions, two mechanisms.

*Regime 1  $\rightarrow$  2.* Prefill is a single pass over the context and is parallel across tokens; it is compute-bound. Decode is sequential—each token depends on the last—and is bound by memory bandwidth, streaming weights and a growing KV cache at every step [1, 7]. Prefill cost therefore scales with context while decode cost per token does not, so prefill eventually overtakes. On hardware that parallelises prefill well this takes a long prompt; on a CPU it happens almost immediately, which is why Platform A appears to start in regime 2.

The datacentre disaggregation literature separates the two phases for exactly this reason [12, 17]; on-device there is nowhere to disaggregate to.

*Regime 2  $\rightarrow$  3.* Unified memory holds weights and KV cache in one pool. As context grows the cache grows with it, and once the combined footprint approaches capacity, decode does not taper—it falls off a cliff, 49 to 17 tok/s in our measurements. Because prefill’s share is a *ratio*, decode getting dramatically slower makes prefill look smaller: the curve turns back down. The reversal is not prefill becoming cheap. It is decode becoming catastrophic. Work on streaming weights from flash addresses a neighbouring constraint [2], but what we observe is a discontinuity on a device a user is holding.

This is why the three are regimes rather than points on a curve. In regime 1 the lever is output length; in regime 2 it is context length; in regime 3 neither helps and the only remedy is to reduce footprint. An optimisation that works in one is neutral or harmful in another.

### 4 WHAT THIS MEANS FOR MOBILE SYSTEMS

1. *Single-number benchmarks conceal every transition.* Reporting “tokens per second” collapses two phases with opposite scaling into one figure, and says nothing about which regime produced it. Two devices reporting the same figure can behave completely differently as a user’s context grows. On-device benchmarks should report prefill and decode separately at minimum, and prefill *fraction against context length* by preference—a curve, not a number. The shape of that curve is the diagnostic: monotonic means two regimes, rise-then-fall means three.

2. *Profiling at one context length is profiling one regime.* Table 2 makes this concrete: the same configuration profiled at 600 and at 2,300 tokens yields opposite optimisation advice. Any measurement reported without its context length is close to meaningless.

3. *Placement and offload decisions inherit the wrong prior.* Routing and cascade systems typically cost a request by expected output length [3, 11, 13]. That is right in regime 1, wrong in regime 2, and dangerously wrong in regime 3, where the true cost is that the request has pushed the device past a cliff it will not recover from until the context shrinks.

4. *The memory wall is a design-time constraint.* Because it is a discontinuity rather than a slope, on-device capability is not gracefully degradable. An assistant that behaves well carrying 1,800 tokens of history may be unusable at 2,200 on the same phone. That determines how much conversation history a product can keep—an architecture decision,

not a tuning flag. And since headroom is what governs it, choosing a smaller model is often better understood as buying regime avoidance than as buying speed.

5. *Context budgeting is the largest available lever in regime 2.* If latency is governed by what the model reads, then what it reads is the thing to control. We demonstrated this on Platform A: gating the costliest optional stage of an agent pipeline—a second “review/verify” model pass—cut end-to-end latency 2.4× with no measurable quality loss under an order-swapped LLM judge [16]. It is the largest single lever on that platform, and it is invisible to anyone optimising decode.

## 5 OPEN QUESTIONS

*Can a runtime detect which regime it is in, and adapt?* This is the question we find most interesting, and the three-regime picture sharpens it into something tractable. The signals are cheap and already available at inference time—prefill fraction, decode throughput, memory headroom—and each regime has a known corresponding action: shorten output in regime 1, budget context in regime 2, cap KV growth or swap to a smaller model in regime 3. A runtime that classified its own regime and switched policy would need no platform-specific tuning from the developer. Whether such a controller is stable near the boundaries, and whether the regime-3 transition can be predicted rather than merely detected after the collapse, we do not know.

*Is there a fourth regime?* Neither platform includes a dedicated neural accelerator. An NPU may add one, or may sit inside an existing regime. If regimes keep accumulating, the argument for measuring rather than assuming strengthens.

*Do architectural changes remove a regime or add one?* Mixture-of-experts alters what must be resident during decode; speculative decoding attacks the sequential dependency that makes decode slow [8]. Both act directly on the mechanisms above. Whether they collapse regimes together or separate them further is empirical, and we have not answered it.

*Limitations.* Each platform is a single machine, with a second CPU vendor as the only cross-device check. We measure three model sizes on Platform B and three families on Platform A, at one primary quantization level each. Absolute figures are characteristic of these devices. The claim we defend is the *structure*: three regimes, two transitions, governed by memory headroom—and that survives across models and across two CPU vendors.

## 6 CONCLUSION

On-device LLM latency has no single characterisation, and not merely because devices differ. The bottleneck moves *within* a device: decode-bound at short context, prefill-bound through the middle, and memory-bound once weights plus KV cache stop fitting—at which point prefill’s share falls not because prefill got cheap but because decode collapsed. Whether a deployment sees one regime, two or three is decided by the headroom a developer leaves, usually without realising it is a decision. Any system that schedules, places or budgets on-device inference—and any benchmark that reports its speed—should measure the hardware and the context length together, rather than inherit an intuition from a datacentre it will never run in.

## REPRODUCIBILITY

Both harnesses, all raw per-run data, and the analysis scripts are public.

- CPU study  
github.com/bapista/cpu-llm-latency  
DOI 10.5281/zenodo.21765705
- Apple Silicon study  
github.com/bapista/mlx-llm-latency  
DOI 10.5281/zenodo.21786211

Every number in this paper traces to a line in a released results file. Earlier versions of both studies are available as preprints at the DOIs above.

## REFERENCES

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 117–134.
- [2] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatami-fard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. LLM in a Flash: Efficient Large Language Model Inference with Limited Memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2312.11514.
- [3] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176* (2023).
- [4] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [5] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *International Conference on Learning Representations (ICLR)*.
- [6] Georgi Gerganov and llama.cpp contributors. 2023. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>.
- [7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model

- Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.
- [8] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.
  - [9] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*.
  - [10] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yinyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Tijmen Blankevoort, and Vikas Chandra. 2024. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. In *International Conference on Machine Learning (ICML)*.
  - [11] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665* (2024).
  - [12] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. arXiv:2311.18677.
  - [13] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident Adaptive Language Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*.
  - [14] Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. 2024. On-Device Language Models: A Comprehensive Review. *arXiv preprint arXiv:2409.00088* (2024).
  - [15] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. 2024. PowerInfer-2: Fast Large Language Model Inference on a Smartphone. *arXiv preprint arXiv:2406.06282* (2024).
  - [16] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
  - [17] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. arXiv:2401.09670.